Asegurar el acceso SSH con autenticación de dos factores y Fail2ban

Javier Siñuka

13 de octubre de 2026

1. Resumen y objetivos

Se realizó un laboratorio de control de acceso SSH, autenticación mediante clave pública y TOTP, y bloqueo temporal con Fail2ban. Se trabajó sobre un **clon aislado** de una máquina virtual Ubuntu; no se modificaron las políticas de ingreso del host. El objetivo fue ejecutar los pasos de las dos guías suministradas [1, 2], observar tanto éxitos como rechazos y conservar evidencia auténtica. Se verificó el acceso por clave, las políticas PAM de denegación y admisión, el segundo factor obligatorio, el ban de una IP cliente, su regla de firewall y el unban. La matriz completa de pruebas está en `evidence/matrix.csv`.

2. Introducción y fundamentos

SSH establece un canal cifrado tras negociar algoritmos y autenticar al servidor. En la autenticación por clave pública, el cliente prueba la posesión de su clave privada mediante una firma; la clave pública autorizada se almacena en el servidor. La frase de paso cifra la clave privada *en el cliente* y no reemplaza la contraseña de la cuenta remota. `ssh-agent` conserva la clave desbloqueada en memoria para evitar repetir la frase de paso en cada conexión. La clave privada usada en esta práctica se creó exclusivamente para el laboratorio, con frase de paso, y permaneció fuera de esta carpeta.

Una clave SSH sigue siendo un solo factor de posesión. MFA agrega otra prueba: con TOTP, un secreto compartido y el tiempo actual producen un código de seis dígitos que cambia normalmente cada 30 segundos. PAM organiza los módulos de autenticación y autorización de Linux por servicio. En este caso, OpenSSH usa PAM para verificar TOTP y, en otra fase separada, para aplicar listas de usuarios. Fail2ban vigila registros, cuenta fallos por origen y ordena al firewall bloquear temporalmente una IP. Es una defensa complementaria contra intentos repetidos y abuso del servicio, con el riesgo de bloquear también a usuarios legítimos que comparten una IP.

3. Entorno y metodología

El host ejecutaba Ubuntu 24.04.5, VirtualBox 7.2.14 y OpenSSH 9.6p1. Se clonó la VM preexistente *Cybersecurity LabVM Workstation*; el clon *Crypto SSH 2FA Fail2ban Lab* ejecutó Ubuntu 22.04.1, kernel 5.15.0-60-generic y OpenSSH 8.9p1. Es una distribución derivada de Debian y permitió usar `apt`, PAM y los servicios pedidos, sin descargar otra imagen. Tenía 2 GiB de memoria y dos CPU virtuales. Se instalaron `libpam-google-authenticator` 20191231-2 y Fail2ban 0.11.2-6.

La VM recibió **10.0.2.15/24** en una interfaz NAT. El host se conectó a `127.0.0.1:2222`, redirigido al puerto 22 del invitado. El servidor registró al cliente como **10.0.2.2**. La consola de VirtualBox quedó como canal de recuperación independiente de SSH. La figura 1 muestra

dirección y estado de los servicios en esa consola. La captura proviene de `VBoxManage controlvm screenshotpng`; se recortó su área vacía solo al incorporarla al documento.

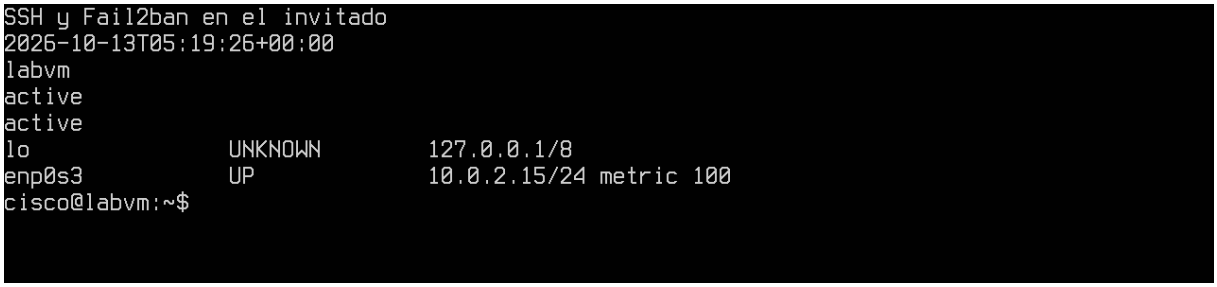


Figura 1: Consola real del clon: dirección IPv4 y servicios SSH y Fail2ban activos.

Se tomaron los snapshots `pre-lab`, `pre-ssh-auth-change`, `pre-pam-access-control` y `pre-mfa-offline`. Un intento de snapshot en vivo previo a MFA avanzó demasiado lentamente; se canceló, se apagó el clon limpiamente y se obtuvo el snapshot `pre-mfa-offline`. Antes de editar SSH/PAM se copiaron `/etc/ssh/sshd_config` y `/etc/pam.d/sshd` a `/root/ssh2fa-backup-2026-10-13/`. Antes de cada recarga se ejecutó `sshd -t`, se inspeccionaron ajustes efectivos con `sshd -T` y se abrió una segunda conexión de prueba.

Las transcripciones en `evidence/logs/` registran comandos y resultados reales. No se incluyeron contraseñas, la clave privada, el secreto TOTP, QR, OTP vigentes ni códigos de recuperación. Se descartó una captura inicial de la pantalla de bienvenida porque contenía credenciales de ejemplo.

4. Desarrollo: control de acceso SSH

4.1. Clave pública, usuario normal y root

Al inicio, `cisco` ingresó con contraseña (salida 0). La salida de `sshd -T` indicó que se admitía contraseña, que la autenticación interactiva estaba desactivada y que regía `PermitRootLogin without-password`. Esta última opción permite potencialmente root por clave autorizada, pero impide su ingreso por contraseña. En nuestro ensayo, root no tenía la clave de laboratorio registrada y su intento terminó con `Permission denied (publickey)`; esto *no* prueba que todo ingreso root por clave estuviera prohibido. También se comprobaron las otras cuentas normales presentes en el clon: `analyst` y `sec_admin` podían usarse localmente, pero SSH las rechazó con la clave de prueba porque no estaba autorizada para ellas.

Se generó una clave Ed25519 exclusiva y se instaló únicamente el archivo `.pub` mediante `ssh-copy-id -i`. La clave privada quedó con modo `0600`, la pública con `0644` y `ssh-agent -l` mostró su huella SHA256. Con `-i` y `IdentitiesOnly=yes`, el servidor aceptó la clave y la sesión devolvió `cisco`; otra clave fue rechazada. Recién después se instaló una configuración con `PasswordAuthentication no`. La clave continuó funcionando y un cliente configurado para usar solo contraseña recibió `Permission denied (publickey)`. Es importante distinguir `/etc/ssh/ssh_config` y `~/.ssh/config`, que configuran al *cliente*, de `/etc/ssh/sshd_config`, que configura al *servidor*. El cliente toma primero las opciones de línea de comandos, luego las de usuario y finalmente las globales [4].

4.2. Algoritmos y vigencia de RSA

El cliente instalado enumeró Ed25519, ECDSA, RSA y tipos con dispositivo de seguridad mediante `ssh -Q key`; esa orden lista capacidades, no necesariamente los algoritmos habilitados por defecto. La enumeración de firmas incluyó `rsa-sha2-256` y `rsa-sha2-512`. RSA sigue siendo

utilizable con esas firmas SHA-2, especialmente por compatibilidad. La firma histórica `ssh-rsa` usa SHA-1 y OpenSSH la deshabilitó por defecto a partir de 8.8; ello no equivale a retirar las claves RSA [3]. Para nuevas claves de este laboratorio se eligió Ed25519. La salida exacta del cliente figura en `evidence/logs/26-host-ssh-algorithms.txt`.

4.3. `nonpriv`, claves centrales y banner

Se creó `nonpriv` con contraseña local aleatoria no registrada. `su - nonpriv -c 'id -un'` funcionó, pero SSH rechazó inicialmente a la cuenta porque no tenía una clave autorizada. Se configuró

```
AuthorizedKeysFile /etc/ssh/authorized-keys/%u .ssh/authorized_keys
```

y se instaló su clave pública en `/etc/ssh/authorized-keys/nonpriv`, con directorio `root:root 0755` y archivo `root:root 0644`. Luego `nonpriv` ingresó por SSH. La ruta central facilita la auditoría y el retiro de claves por un administrador, a costa de gestionar permisos y altas de forma centralizada. `AuthorizedKeysCommand` y `AuthorizedKeysCommandUser` permiten consultar claves desde un programa y una identidad restringida; requieren controlar ese programa, su origen de datos y los registros de auditoría. No se conectó LDAP en esta práctica [4].

Se agregó un banner académico en `/etc/ssh/ssh2fa-banner` y la directiva `Banner` en `/etc/ssh/sshd_config.d/30-banner.conf`. El cliente mostró el texto antes de completar la autenticación. La ruta `/etc/sshd_config` mencionada en la guía no era la del servidor instalado.

4.4. Listas de acceso PAM

Se probaron dos estados distintos de `/etc/pam.d/sshd`. Primero, una regla `account required pam_listfile.so` con `sense=deny` rechazó a `nonpriv` y dejó entrar a `cisco`. Después, al sustituirla por `sense=allow` y listar solamente `cisco`, este entró y `nonpriv` fue rechazado. Los rechazos quedaron registrados como `pam_listfile(sshd:account): Refused user nonpriv`. Se usó la fase `account` para que la comprobación también ocurriera tras una clave pública válida. Se eligió `onerr=fail` para evitar permitir el ingreso si falta el archivo de política. Cada lista tuvo dueño root y modo `0600`. Antes de MFA, se restauró el PAM original para no mezclar políticas y confundir la causa de cada resultado.

5. Desarrollo: autenticación de dos factores

Como `cisco`, se ejecutó `google-authenticator` en modo TOTP, con prohibición de reutilizar códigos, ventana de tres pasos temporales y límite de tres intentos cada 30 segundos. El archivo `~/.google_authenticator` quedó propiedad del usuario y modo `0400`. Los códigos de recuperación quedaron dentro de ese archivo; no se imprimieron ni copiaron al repositorio. Una instalación real debe custodiarlos de forma separada y segura.

Se respaldaron de nuevo PAM y la configuración de SSH. La fase probada usó

```
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication yes
UsePAM yes
AuthenticationMethods publickey,keyboard-interactive
```

En PAM se comentó `@include common-auth` para no pedir además la contraseña local y se agregó `auth required pam_google_authenticator.so`. No se usó `nullok`; el usuario protegido no podía omitir TOTP. La figura 2 muestra los archivos reales. Configurar solo `PasswordAuthentication no` no habría bastado para desactivar otros métodos interactivos. `AuthenticationMethods` exige ambas etapas en el orden indicado [4].

```
cisco@labvm:~$ date -Is; cat /etc/ssh/sshd_config.d/10-lab-ssh.conf; sed -n "1,8p" /etc/pam
.d/sshd
2026-10-13T05:19:57+00:00
# SSH lab final phase: public key followed by PAM keyboard-interactive TOTP.
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication yes
UsePAM yes
PermitRootLogin prohibit-password
AuthenticationMethods publickey,keyboard-interactive
# PAM configuration for the Secure Shell service

# Standard Un*x authentication.
#@include common-auth
auth required pam_google_authenticator.so

# Disallow non-root logins when /etc/nologin exists.
account    required     pam_nologin.so
cisco@labvm:~$
```

Figura 2: Configuración efectiva de la fase MFA: clave pública seguida de PAM/TOTP.

No hubo un teléfono enrolado. Para comprobar el paso criptográfico se usó `scripts/totp_local_client.py` **dentro de la VM**: lee el secreto localmente, calcula HMAC-SHA1/TOTP y entrega el código a una segunda sesión SSH, sin imprimirlo. Es una emulación de laboratorio y no demuestra separación física de dispositivos. Una clave válida más un código actual abrió la sesión (salida 0 y marcador de comando observado), como muestra la figura 3. Un código incorrecto y uno vencido terminaron con salida 255; una clave incorrecta fue rechazada antes del prompt TOTP. Desde el host, la clave correcta sin código fue aceptada en la primera etapa, pero SSH continuó solicitando `keyboard-interactive` y denegó la sesión. El registro del servidor mostró `Accepted google_authenticator` para los éxitos y `Invalid verification code` para los fallos.

```
cisco@labvm:~$ date -Is; python3 ~/totp_local_client.py valid
2026-10-13T05:29:04+00:00
mode=valid
otp_prompt_seen=true
command_marker_seen=true
ssh_exit=0
cisco@labvm:~$ _
```

Figura 3: Sesión SSH local de prueba completada con clave y TOTP actual; el código no se muestra.

```
cisco@labvm:~$ date -Is; python3 ~/totp_local_client.py wrong
2026-10-13T05:21:07+00:00
mode=wrong
otp_prompt_seen=true
command_marker_seen=false
ssh_exit=255
cisco@labvm:~$ _
```

Figura 4: Código TOTP incorrecto: se solicita el segundo factor, pero no se ejecuta el comando remoto.

Si se pierde el dispositivo, la consola de VirtualBox y un snapshot permiten recuperar el clon sin cambiar la política del host. En producción deben custodiarse códigos de recuperación de un solo uso y re-enrolar de forma controlada si se pierde el secreto. `nullok` permite transiciones o excepciones por cuenta, pero deja a esas cuentas con un solo factor; no se aplicó aquí. El archivo por usuario puede controlar reutilización, ventana temporal, límite de intentos y códigos de emergencia; ampliar la ventana o permitir reutilización reduce la resistencia ante códigos capturados. La salida real de `google-authenticator -help` mostró las opciones `-d`, `-w`, `-r`, `-R`, `-e` y `-C` para una inscripción automatizada. La guía también propone un tercer factor: reactivar `common-auth` haría que PAM pidiese contraseña local además de TOTP después de la clave SSH. Estas variantes se explican, pero no se activaron ni se presentan como resultados experimentales.

6. Desarrollo: Fail2ban

Se instaló Fail2ban y se configuró un jail `[sshd]` en `/etc/fail2ban/jail.d/sshd.local` con `enabled=true`, `bantime=10m`, `findtime=10m` y `maxretry=5`. La opción correcta es `enabled`, no `enable` como aparece en la guía. En esta VM existía `/var/log/auth.log` y `rsyslog` estaba activo; se eligió `backend=polling`. El firewall usó `iptables-multiport` sobre `iptables v1.8.7 (nf_tables)`. Después de un breve inicio del servicio, `fail2ban-client status sshd` mostró el archivo vigilado y ninguna IP baneada.

Se generaron fallos controlados exclusivamente contra el clon desde el host. Al arrancar el jail se habían registrado tres fallos MFA recientes desde `10.0.2.2`; Fail2ban también los detectó. Dos intentos de usuario inválido posteriores completaron el umbral de cinco dentro de diez minutos; un sexto evento se registró prácticamente a la vez que el ban. El registro mostró `Ban 10.0.2.2`. `iptables -L` confirmó el salto de `INPUT` a `f2b-sshd` para el puerto 22 y una regla `REJECT` para esa IP. Un cliente autorizado desde el host quedó bloqueado antes del banner y agotó el tiempo de espera. El segundo ban controlado permitió capturar en pantalla la regla activa de la figura 5.

```
cisco@labvm:~$ date -Is; sudo fail2ban-client status sshd; sudo iptables -L f2b-sshd -n
2026-10-13T05:22:01+00:00
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     10
|  `- File list:        /var/log/auth.log
`- Actions
   |- Currently banned: 1
   |- Total banned:     2
   `- Banned IP list:   10.0.2.2
Chain f2b-sshd (1 references)
target     prot opt source               destination
REJECT     all  --  10.0.2.2            0.0.0.0/0           reject-with icmp-port-unreach
able
RETURN     all  --  0.0.0.0/0           0.0.0.0/0
cisco@labvm:~$
```

Figura 5: Fail2ban registra la IP baneada y `iptables` contiene la regla de rechazo.

Desde la consola se ejecutó `fail2ban-client set sshd unbanip 10.0.2.2`. El jail pasó a cero IP baneadas y la regla `REJECT` desapareció (figura 6). Tras el unban, el host volvió a alcanzar la etapa TOTP: el servidor aceptó la clave y ofreció `keyboard-interactive`. No se completó un ingreso desde el host porque el secreto TOTP permaneció dentro del clon. El cliente local sí completó de nuevo MFA. Esto distingue recuperación de red desde la IP antes baneada de una autenticación completa desde ese cliente.



Figura 6: Desbloqueo manual: lista vacía de IP baneadas y ausencia de la regla `REJECT`.

Fail2ban también puede vigilar otros servicios con filtros adecuados, por ejemplo servidores web, correo y servicios de archivos; sus acciones pueden enviar avisos por correo además de modificar el firewall [5]. Depende de registros correctos y de una acción de firewall efectiva. Un ban por IP puede afectar a usuarios legítimos detrás del mismo NAT y es menos eficaz ante ataques distribuidos. En este ensayo, la consola de VirtualBox permitió desbanear sin depender de la IP bloqueada.

7. Resultados y análisis

La tabla 1 resume los resultados observados. *Pasa* indica que el comportamiento coincidió con lo esperado; *parcial* explicita el límite de la prueba desde el host. El CSV adjunto incluye procedimiento, fase de configuración, origen y archivo de evidencia por fila.

Cuadro 1: Matriz resumida de pruebas ejecutadas.

ID	Escenario	Esperado	Observado
T01	VM alcanzable y SSH activo	Servicio activo	`10.0.2.15`; activo
T02	Usuario normal antes de claves	Ingreso con contraseña	`cisco`; salida 0
T03	Root con clave no autorizada	Rechazo	Salida 255; clave ausente
T04	Clave pública autorizada	Ingreso	`cisco`; salida 0
T05	Clave equivocada	Rechazo	`Permission denied`
T06	Solo contraseña tras desactivarla	Rechazo	Salida 255; clave sigue válida
T07	`nonpriv` antes de clave	Local sí; SSH no	`su` sí; SSH salida 255
T08	`nonpriv` con clave central	Ingreso SSH	`nonpriv`; salida 0
T09	Banner SSH	Visible antes de ingreso	Texto ASCII observado
T10	PAM denylist	Rechazar listado	`nonpriv` no; `cisco` sí
T11	PAM allowlist	Admitir listado	`cisco` sí; `nonpriv` no
T12	Clave y TOTP actual	Ingreso	Marcador y salida 0
T13	Clave válida sin TOTP	Rechazo	Pide segundo factor; salida 255
T14	TOTP incorrecto y vencido	Rechazo	Ambos salida 255
T15	Clave errónea durante MFA	Rechazo previo a TOTP	Sin prompt; salida 255
T16	Jail SSH de Fail2ban	Activo	`sshd`; `auth.log` vigilado
T17	Umbral y regla de bloqueo	Ban de IP cliente	`10.0.2.2`; `REJECT`
T18	Conexión tras ban	Bloqueo	Timeout antes del banner
T19	Unban y reconexión	Recuperación	Host vuelve a TOTP; MFA local sí (parcial)
T20	Tercer factor opcional	Opcional	No ejecutado

El primer `fail2ban-client status` se lanzó inmediatamente tras iniciar el servicio y aún no encontró su socket; la siguiente consulta sí devolvió el jail activo. Los fallos que contaron para el primer ban incluyeron pruebas MFA previas. En las pruebas TOTP negativas iniciales el cliente de prueba quedó esperando un segundo prompt; se ajustó `NumberOfPasswordPrompts=1` y se repitieron. Los resultados definitivos son los de `evidence/logs/33-mfa-totp-retake.txt`. Estos detalles se conservan para no presentar una secuencia más limpia que la observada.

8. Discusión y respuestas a las guías

La lista de denegación sirve cuando casi todos los usuarios pueden entrar y solo unos pocos son excepciones. La lista de admisión es más fácil de auditar cuando solo un grupo reducido debe tener acceso remoto; una cuenta puede seguir siendo válida en la consola local y ser rechazada por SSH. En ambos casos se deben gestionar cambios de personal, propiedad de archivos y recuperación ante errores. La política PAM de tipo `account` aplicada aquí se verificó también con claves públicas.

El ensayo de root mostró la diferencia entre la política efectiva y una prueba concreta: `without-password` no equivale a `PermitRootLogin no`. El usuario `nonpriv` falló primero porque no tenía clave autorizada, no por carecer de contraseña local. La ruta central de claves permitió autorizarlo; en un servicio real habría que revocar claves obsoletas y registrar cada alta.

Tras los fallos, Fail2ban creó una regla para la IP cliente y bloqueó nuevas conexiones al puerto SSH. `iptables -L` exhibió la cadena `f2b-sshd` y el `REJECT`; el unban retiró esa entrada. Otros filtros pueden vigilar servicios web o de correo. Una acción de aviso por correo es posible, pero no se configuró ni verificó un envío en esta práctica. El modo de tercer factor, las excepciones por cuenta y la automatización de enrolamiento quedaron como opciones explicadas, sin atribuirles pruebas.

9. Conclusiones

El laboratorio demostró que se puede exigir una clave SSH y un segundo paso TOTP, restringir cuentas mediante PAM y bloquear temporalmente la IP de un cliente que acumula fallos. Las pruebas negativas fueron tan importantes como las positivas: detectaron el efecto real de cada configuración y permitieron revertirla antes de la fase siguiente. Los snapshots, las copias de archivos, la validación con `sshd -t` y la consola separada evitaron perder el acceso durante los cambios.

La principal limitación es que el TOTP válido se generó dentro de la VM con un cliente emulado; se verificó la secuencia criptográfica y de OpenSSH, pero no la custodia en un teléfono independiente. Después del unban, el host recuperó conectividad hasta la solicitud del segundo factor, mientras el cliente local completó MFA. El modo opcional de tres factores y las alertas de Fail2ban no fueron ejecutados. Las configuraciones y scripts de esta carpeta permiten repetir el ensayo o restaurar el clon. Las claves SSH son apropiadas para administrar cuentas remotas, PAM permite definir quién puede entrar y TOTP agrega protección cuando una clave podría ser robada. Fail2ban aporta control de frecuencia para un servicio expuesto, siempre que se vigilen los registros y se conserve una vía de recuperación.

10. Anexo: archivos y recuperación

- `configs/`: directivas SSH, reglas PAM, banner y jail de Fail2ban.
- `scripts/guest-phase.sh`: preparación, fases separadas y rollback dentro de un invitado de laboratorio.
- `scripts/totp_local_client.py`: cliente TOTP local que no imprime secretos ni códigos.
- `evidence/matrix.csv`: procedimientos, resultados y nombres de evidencia. `evidence/logs/` contiene transcripciones; las capturas reales están en `evidence/screenshots/`.

Para restaurar el clon ejecutado se dispone de la consola y de las copias en `/root/ssh2fa-backup-2026-10-13/`. El procedimiento reproducible y la invocación de rollback figuran en `README.md`. No se debe reutilizar ninguna credencial de este clon en un sistema de producción.

Referencias

[1] Material de la actividad, *Control de acceso en ssh sin todas las vm*. PDF suministrado en esta carpeta.

[2] Material de la actividad, *Copia de Two-factor authentication (2FA)*. PDF suministrado en esta carpeta.

[3] OpenSSH, *Release Notes*, sección OpenSSH 8.8: transición de firmas `ssh-rsa` a RSA/SHA-2. https://www.openssh.com/releasenotes.html. Consulta: 13 de octubre de 2026.

[4] OpenBSD, *sshd_config(5)* y *ssh_config(5)*. https://man.openbsd.org/sshd_config.5 y https://man.openbsd.org/ssh_config.5. Consulta: 13 de octubre de 2026.

[5] Fail2ban, *jail.conf* y acciones, repositorio oficial. https://github.com/fail2ban/fail2ban/blob/master/config/jail.conf. Consulta: 13 de octubre de 2026.